

Machine Learning

Feature Importance and Model Explainability



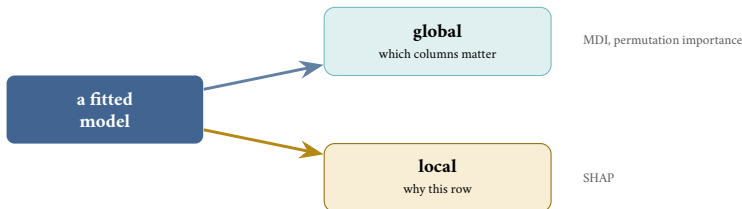
Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

Machine Learning (25SC2107E)

Session Outcome

- Be able to say which features a model used (MDI, permutation importance) and why one row got its answer (SHAP) and know what each of them gets wrong.



Model Explainability

Key Terms — Explaining a Model

Definitions

Interpretability — how far a person can follow how a model reaches its answers. A short decision tree is interpretable; a 500-tree forest is not.

Explainability — attaching an after the fact explanation to a model that is not itself interpretable.

Feature importance — a number saying how much a model relies on a given column.

Global explanation — one answer for the **whole dataset**: which columns matter overall (MDI, permutation importance).

Local explanation — one answer for a **single row**: why this student got this prediction (SHAP).

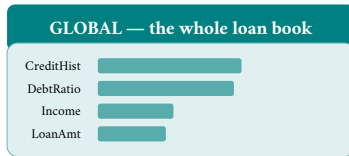
Feature Importance and Model Explainability

Why Explain a model?

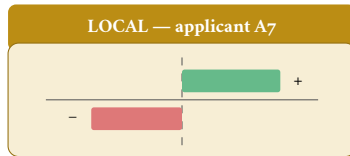
Suppose you build a model that predicts whether a loan applicant will default. It works well, ninety percent accuracy. Then the applicant asks why they were rejected, or your manager asks which factors the model relies on, or a regulator asks whether the model is using something it should not.

Accuracy alone cannot answer these questions. You need to know what the model is actually paying attention to. That is what feature importance and explainability are for. There are two different questions

- *Global explanation* asks which features matter to the model overall, across all the data.
 - MDI and permutation importance answers this question
- *Local explanation* asks why this particular row got this particular prediction.
 - SHAP (SHapley Additive exPlanations) answers this question.



"credit history matters more than the loan amount"



"high debt ratio pushed this down by 0.18"

MDI — Mean Decrease in Impurity

Key Terms — Impurity and MDI

Definitions

Impurity — how mixed the classes are in a node. Gini and entropy are the usual measures; a pure node (one class only) has impurity 0.

Impurity decrease — how much a split lowers impurity, weighted by the fraction of rows that reached that node.

MDI (mean decrease in impurity) — a feature's importance, obtained by adding up every impurity decrease it produced, across every node of every tree, then normalising to sum to 1.

Cardinality — the number of distinct values a column takes. A high-cardinality column offers many candidate split points.

Cardinality bias — MDI's tendency to over-rate high-cardinality columns, because with many thresholds to try, one will look good by chance on the training rows.

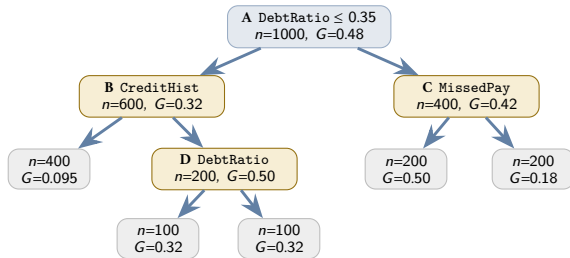
Mean Decrease in Impurity (MDI)

Recall how a tree builds a split. It picks a feature and a cut point, and it chooses the one that reduces the error the most. That reduction is a number the tree already calculated during training.

- Every time a feature is used for a split anywhere in the tree, we note how much error that split removed, weighted by how many rows passed through that node.
- MDI simply collects and adds those numbers for a feature
- Do this across all trees in the forest or the boosted ensemble, and average.
- The numbers are already collected during training, so `model.feature_importances_` in scikit-learn or LightGBM returns instantly. For a quick first look at a model, this is genuinely useful.

How MDI Is Computed — One Tree, All the Numbers

One tree of the loan forest. $N = 1000$ applications, 400 defaulted, so $G_{\text{root}} = 1 - 0.4^2 - 0.6^2 = 0.48$. Grey = leaf.



$$\text{MDI}(j) = \sum_{\text{nodes splitting on } j} \frac{n_{\text{node}}}{N} \times (\text{impurity drop})$$

$$\text{Node A drop: } 0.48 - 0.6(0.32) - 0.4(0.42) = \mathbf{0.12}$$

Node	Feature	n	drop	$\frac{n}{N} \times \text{drop}$
A	DebtRatio	1000	0.120	0.120
B	CreditHist	600	0.090	0.054
C	MissedPay	400	0.080	0.032
D	DebtRatio	200	0.180	0.036

Add up per feature, then normalise to sum to 1

DebtRatio $0.120 + 0.036 = 0.156 \rightarrow \mathbf{0.645}$

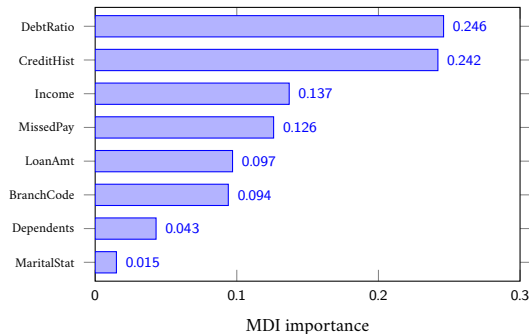
CreditHist $0.054 \rightarrow \mathbf{0.223}$

MissedPay $0.032 \rightarrow \mathbf{0.132}$

(total 0.242)

DebtRatio is split on twice, so both nodes count. These three numbers are `model.feature_importances_` — free, and averaged over all trees in the forest.

MDI on Our Loan Forest



✓ DebtRatio, CreditHist on top — correct

✓ MaritalStat near zero — it is noise

✗ BranchCode (0.094) carries **no signal**, yet it outranks Dependents, a real feature

Challenges in MDI

Consider two features, a customer's city (say 8 categories) and their exact account balance (thousands of distinct values). The balance feature has far more possible cut points, so it gets far more opportunities to be selected as a split.

- The result is that MDI is biased towards high-cardinality features (features with many distinct values).
 - Continuous variables and ID-like columns get higher importance, while binary and low-category features get understated.
- MDI is computed on the training data only. If the model overfits and uses a noisy feature to memorise training rows, MDI will report that feature as important, even though it contributes nothing on new data.

Permutation Importance

Key Terms — Permutation Importance

Definitions

Permutation importance — the drop in score when one column's values are randomly shuffled, breaking its link to the target while leaving every other column untouched.

Shuffling (permuting) — reordering a column's values among the rows. Nothing is deleted and the model is not refitted.

Held-out data — rows the model was not trained on. Permutation importance is measured here, so it asks whether a column helps the model generalise.

Negative importance — a score below 0, meaning shuffling slightly helped. It is noise, and should be read as “no importance”.

Group permutation importance — shuffling a set of correlated columns together, because shuffling them one at a time makes each look useless.

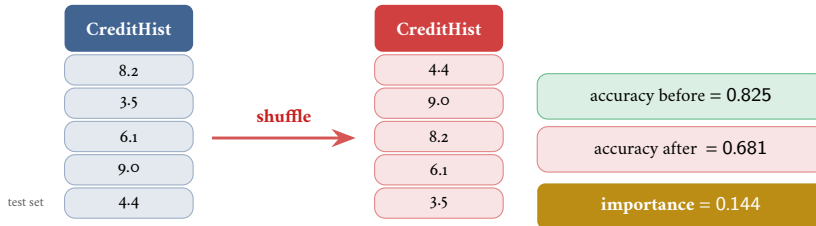
Permutation Importance

Permutation importance approaches the problem from the opposite direction. Instead of looking inside the model, it treats the model as a black box and asks a simple experimental question: if this feature is removed, how much worse would the predictions get? Procedure

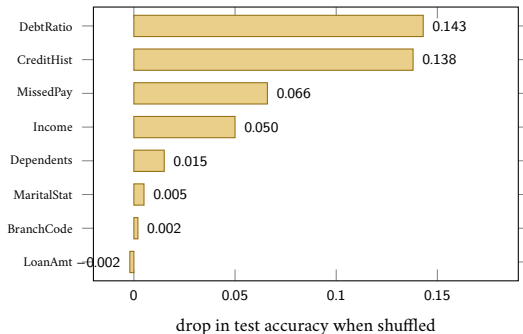
- Measure the model's score on held-out test data. Call it s_0
- Take one feature column and shuffle its values randomly among the rows. This breaks any relationship between that feature and the target, while keeping its distribution intact.
- Score the model again on this corrupted data. Call it s_1
- The importance of that feature is the drop: $s_0 - s_1$
- Restore the column, and repeat for every feature.

If shuffling a feature causes accuracy to collapse, the model clearly depended on it. If shuffling changes nothing, the model was ignoring it.

Permutation Importance: Shuffle one column



Permutation Importance on Loan Forest



✓ DebtRatio, CreditHist lead by a clear margin

✓ BranchCode falls from 0.094 (MDI) to 0.002 — the fake importance is gone

✓ Dependents now outranks BranchCode

LoanAmt = -0.002: a **negative** value is noise — read it as “no importance”

MDI or Permutation Importance

	MDI (impurity)	Permutation
Computed on	training data	test data
Cost	free	one pass per feature
Works for	tree models only	any model
Biased to many-valued columns	yes	no
Can be negative	no	yes (means “no use”)
scikit-learn	<code>feature_importances_</code>	<code>permutation_importance</code>

Two advantages over MDI

- It works on held-out data, so it measures importance for real predictive performance rather than for fitting the training set
- It is *model-agnostic*, since the procedure never looks inside the model, so it works the same way for a random forest, an XGBoost model, a linear regression or a neural network, making it useful for comparing across model types.

Code

```

1 from sklearn.inspection import \
2     permutation_importance
3
4 # 1. the free one (training data, biased)
5 for n, v in zip(NAMES, rf.feature_importances_):
6     print(f"{n:10s} {v:.3f}")
7
8 # 2. the honest one (test data)
9 pi = permutation_importance(
10     rf, Xte, yte,
11     n_repeats=30,
12     random_state=0,
13     n_jobs=-1)
14
15 for i in pi.importances_mean.argsort()[::-1]:
16     print(f"{NAMES[i]:10s} "
17           f"{pi.importances_mean[i]:+.3f}")

```

- `n_repeats=30` — more repeats, steadier numbers.
- Always print the standard deviation too: if it is as large as the mean, the ranking is not real.
- Pass `Xte`, `yte` — using training data brings the MDI problem back.

`scoring=` lets you use any metric, not just accuracy.

SHAP — SHapley Additive exPlanations

Key Terms — SHAP

Definitions

Shapley value — from game theory (Shapley, 1953): a fair way to split a payout among players, by averaging each player's marginal contribution over all the orders in which they could join.

Coalition — a subset S of the features, treated as the ones we “know”.

Marginal contribution — how much adding feature j to a coalition changes the prediction.

Base value $\mathbb{E}[f]$ — the average prediction over everybody: what we would say knowing nothing about this row.

SHAP value ϕ_j — how far feature j pushed this row's prediction away from the base value. Positive pushes up, negative pushes down.

Additivity — the guarantee that $f(x) = \mathbb{E}[f] + \phi_1 + \dots + \phi_p$ exactly. A feature the model ignores gets $\phi_j = 0$.

TreeSHAP — an exact, fast algorithm for computing SHAP values on tree models.

SHAP

SHAP, which stands for SHapley Additive exPlanations, answers the local question. For a single prediction, it splits that prediction into contributions, one per feature.

- MDI and Permutation importance give you one number per feature for the whole model.
- Neither can tell the rejected loan applicant anything, because "income is the most important feature overall" says nothing about their application.

For any one row, the contributions add back up to the prediction exactly:

$$\hat{y} = \mathbb{E}[f] + \sum_{j=1}^P \phi_j$$

In the loan approval case

$$\underbrace{\hat{y}}_{\text{this applicant's score}} = \underbrace{\hat{y}_{\text{base}}}_{\text{base value (average applicant)}} + \phi_{\text{Job}} + \phi_{\text{OwnsHouse}} + \phi_{\text{CreditHist}} + \dots$$

SHAP in Code

```
1 # pip install shap
2 import shap
3
4 explainer = shap.TreeExplainer(rf)
5 shap_values = explainer.shap_values(Xte)
6
7 # one student, one explanation
8 shap.force_plot(explainer.expected_value[1],
9                 shap_values[1][0], Xte[0],
10                 feature_names=NAMES)
11
12 # global ranking, from local values
13 shap.summary_plot(shap_values[1], Xte,
14                  feature_names=NAMES,
15                  plot_type="bar")
```

- TreeExplainer — exact and fast for trees, forests, XGBoost, LightGBM.
- expected_value is the base value; the SHAP values add on top.
- force_plot explains **one** row; summary_plot averages over many.
- Non-tree models: KernelExplainer, much slower.

SHAP explains the **model**, not the world. It is not proof of cause.

Tree Models and Neural Networks

Trees explain themselves

Follow any path from root to leaf and you get a plain list of conditions: income below 40 thousand, employment under 2 years, loans above 3, therefore high risk. Even in a large ensemble, where no single path can be read, split gains give MDI for free and tree structure gives exact SHAP values efficiently.

Networks do not

A network spreads what it learned across thousands of weights, none of which mean anything on their own, so there is nothing to read. Explanation must be added from outside, using gradient methods or local surrogates like LIME. These approximate the model rather than read it, and different methods can disagree on the same prediction.

What this means for tables

Boosted trees usually match or beat networks on tabular data anyway, and come with faster, exact explanations, which matters when a decision affects a person. Networks earn their place on images, audio and text, where explanation is often less central.

Trees or Neural Networks?

Tree models

best on tables

no scaling needed

categoricals handled directly

works on small data

cheap to train

cannot extrapolate

Neural networks

best on images, text, audio

scaling required

categoricals need encoding

needs a lot of data

expensive to train

extrapolates, to a degree

On our 800 students: random forest 0.825 vs neural network (32, 16) 0.787

- ❶ MDI is computed on
(a) the test set (b) the training set (c) a fresh sample (d) the validation set **Answer: (b)**
- ❷ MDI tends to over-rate features that have
(a) few distinct values (b) many distinct values (c) missing values (d) negative values **Answer: (b)**
- ❸ Permutation importance works by
(a) removing a column and refitting (b) shuffling a column and re-scoring (c) setting a column to zero (d) growing more trees **Answer: (b)**
- ❹ A permutation importance of -0.004 means the feature is
(a) very important (b) harmful and should be removed (c) of no use to the model (d) miscoded **Answer: (c)**

- 5 SHAP values for one row must add up to
(a) 1 (b) 0 (c) prediction minus base value (d) the accuracy
- 6 A feature that the model never uses receives a SHAP value of
(a) exactly 0 (b) a small positive number (c) $1/p$ (d) undefined
- 7 Which method gives a *local* explanation?
(a) MDI (b) permutation importance (c) SHAP (d) all three
- 8 For a table of numbers and categories, the usual strongest model is
(a) a neural network (b) gradient-boosted trees (c) *k*-means (d) linear regression

Answer: (c)**Answer: (a)****Answer: (c)****Answer: (b)**